TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC



BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

Bài thực hành L02

MapReduce với Hadoop Streaming, map-side join và combiner

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

1 Mục tiêu và môi trường **2**

2 Thực hiện và kết quả **3**
2.1 Đếm từ, kiểm thử cục bộ . . . 3
2.2 Đếm từ trên YARN . . . 4
2.3 Chuẩn bị dữ liệu taxi và map-side join . . . 5
2.4 Có và không có combiner . . . 6
2.5 Giao diện YARN, JobHistory và CLI . . . 8
2.6 Mở rộng: giá cước trung bình theo khu vực . . . 13

3 Trả lời câu hỏi checkpoint **14**
3.1 Trong đường ống cục bộ, lệnh nào làm việc của shuffle, và thiếu nó thì sao? . . . 14
3.2 Vì sao reducer Streaming phải tự giữ `current_key`? . . . 14
3.3 Job taxi có bao nhiêu map task, bao nhiêu reduce task, và vì sao? . . . 14
3.4 Combiner giảm `Reduce shuffle bytes` bao nhiêu lần, và vì sao đầu ra combiner xấp xỉ số khu vực nhân số map task? . . . 14
3.5 Vì sao combiner không làm job nhanh hơn ở đây, và khi nào nó có ích rõ? . . . 15
3.6 Có dùng lại reducer "trung bình" làm combiner được không? . . . 15
3.7 Map-side join trong bài hoạt động thế nào, và khi nào nó không còn hợp? . . . 15
3.8 Counter `BAD_RECORDS` hiện ở đâu, và vì sao nên đếm dòng hỏng thay vì cho job dừng? . . . 15

4 Kết luận **16**

Tài liệu tham khảo **16**

Phụ lục: Trích đoạn mã nguồn **16**

Danh mục hình

1 Kiểm thử cục bộ đường ống `cat | mapper | sort | reducer` trên bốn cuốn sách: top 5 từ và số dòng `the` khi bỏ `sort`. . . . 4
2 Counter cuối job `wordcount` trên YARN: 4 map task, `Reduce input groups=20967` và top 5 từ đọc từ HDFS. . . . 5
3 Chuyển Parquet sang CSV, 6 block trên HDFS, kiểm thử cục bộ map-side join trên 1,000 dòng và counter `BAD_RECORDS`. . . . 6
4 Trích các counter so sánh từ hai log và top 10 khu vực đón khách; `diff` giữa hai đầu ra cho kết quả IDENTICAL. . . . 8
5 YARN ResourceManager UI (cổng 8088): ba ứng dụng MapReduce đã hoàn thành và các chỉ số cluster. . . . 9

6 JobHistory UI (cổng 19888): danh sách các job đã hoàn thành. 9
7 JobHistory UI: tổng quan job `zones_combiner` với 6 map task, 2 reduce task và thời gian từng pha. 10
8 JobHistory UI: trang Counters của job không combiner. 11
9 JobHistory UI: trang Counters của job có combiner. 12
10 Kiểm tra các job đã hoàn thành bằng `yarn application -list` và `mapred job -list all`. 13
11 Job tính giá cước trung bình theo khu vực với combiner (`sum, count`) và đối chiếu JFK với bản chạy không combiner. 13

Danh mục bảng

1 Môi trường thực nghiệm. 3
2 Dữ liệu sử dụng trong thực nghiệm. 3
3 Counter của job đếm từ trên YARN. 4
4 So sánh counter giữa hai lần chạy không và có combiner. 7
5 Counter bổ sung của hai lần chạy khu vực (trích từ `nocombiner.log` và `combiner.log`). 7
6 Giá cước trung bình (`fare_amount`) của 10 khu vực đón khách nhiều chuyến nhất. 13
7 Danh mục tệp nộp kèm trong `submission/L02/`. 16

1 Mục tiêu và môi trường

Bài thực hành viết mapper và reducer bằng Python, thử bằng đường ống Unix `cat | mapper | sort | reducer`, rồi chạy trên YARN qua Hadoop Streaming [2]. Có hai bài toán: đếm từ trên bốn cuốn sách Project Gutenberg, và đếm số chuyến taxi theo khu vực đón khách, trong đó mã khu vực được đổi sang tên bằng map-side join qua distributed cache (`-files`). Thí nghiệm chính so sánh counter của cùng một job khi có và không có combiner. Sau đó các job được xem lại trên ResourceManager UI (cổng 8088), JobHistory UI (cổng 19888) và CLI. Phần mở rộng tính giá cước trung bình theo khu vực.

Bài dùng lại cluster Docker của L01 (Bảng 1). Thư mục bài được gắn vào `/lab/L02`, mọi lệnh chạy trong container `namenode` vì Streaming cần `python3` có mặt trên nút chạy task. Dữ liệu ở Bảng 2: bốn cuốn sách có mã 1342, 11, 84 và 2701, và 1,000,000 dòng đầu của tệp Parquet `yellow_tripdata_2024-01`, chuyển sang CSV không tiêu đề bằng `parquet_to_csv.py` vì Streaming chỉ đọc văn bản.

Bảng 1. Môi trường thực nghiệm.

Thành phần	Cấu hình
HDFS	1 NameNode, 3 DataNode, replication factor = 3
YARN	1 ResourceManager (Capacity Scheduler), 1 NodeManager (1 Active Node trên UI)
Tài nguyên YARN	NodeManager cấp 1.5 GB: AM 512 MB + 2 task 384 MB chạy đồng thời
Phân bổ container	tối thiểu `<memory:128, vCores:1>`, tối đa `<memory:1536, vCores:4>`
Lịch sử job	JobHistory Server, cổng 19888; ResourceManager UI cổng 8088
Ngôn ngữ mapper/reducer	Python 3, chạy qua `hadoop-streaming-*.jar`

Bảng 2. Dữ liệu sử dụng trong thực nghiệm.

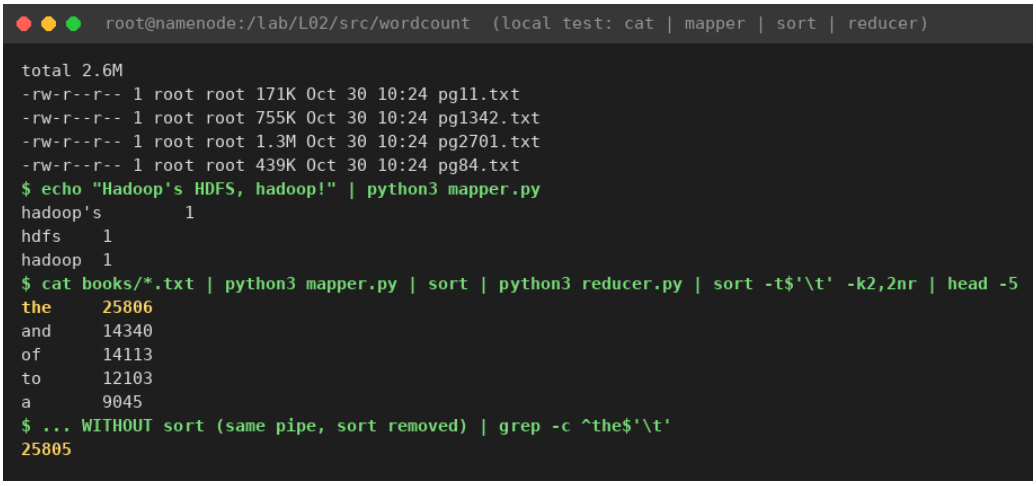
Bộ dữ liệu	Kích thước	Số bản ghi	Lưu trữ HDFS
Sách Gutenberg (4 tệp)	pg11 171K, pg1342 755K, pg2701 1.3M, pg84 439K (tổng 2.6M)	48,749 dòng	`/user/root/books`, 4 tệp
Taxi CSV 2024-01	92,847,108 byte (92.8 MB)	1,000,000 dòng	`/user/root/taxi_csv`, block 16 MB, 6 block
`taxi_zone_lookup.csv`	nhỏ	265 dòng	gửi qua `-files`

2 Thực hiện và kết quả

2.1 Đếm từ, kiểm thử cục bộ

Với Streaming, mapper và reducer đọc dòng văn bản từ stdin và in `key<TAB>value` ra stdout. Mapper đổi văn bản sang chữ thường rồi lấy từ theo biểu thức `[a-z][a-z']*`; với đầu vào `Hadoop's HDFS, hadoop!` nó in ra ba cặp `hadoop's 1`, `hdfs 1`, `hadoop 1`. Reducer tự nhận ra chỗ đổi khóa trên luồng đã sắp xếp và in thêm khóa cuối sau vòng lặp (xem Phụ lục). Chạy cả đường ống trên bốn cuốn sách, năm từ nhiều nhất là `the 25806`, `and 14340`, `of 14113`, `to 12103` và `a 9045`, đúng giá trị đề bài cho. Khi bỏ `sort`, reducer in ra 25,805 dòng bắt đầu bằng

`the` thay vì một dòng tổng (Hình 1): gần như lần xuất hiện nào cũng thành một nhóm riêng, chỉ những chỗ `the` tình cờ đứng liền nhau mới được gộp.



Hình 1. Kiểm thử cục bộ đường ống `cat | mapper | sort | reducer` trên bốn cuốn sách: top 5 từ và số dòng `the` khi bỏ `sort`.

2.2 Đếm từ trên YARN

Sách được đưa lên `/user/root/books` và job chạy bằng `hadoop jar $STREAM` với `-files mapper.py,reducer.py` để gửi script tới thư mục làm việc của từng task. Job `wordcount` (`job_1793355278127_0001`) chạy thành công với `number of splits:4`: mỗi cuốn sách nhỏ hơn một block nên được một split, vì vậy có 4 map task và 1 reduce task theo mặc định (Bảng 3, Hình 2). Số khóa vào reducer bằng số dòng đầu ra, và top 5 đọc từ HDFS giống bản chạy cục bộ.

Bảng 3. Counter của job đếm từ trên YARN.

Counter	Giá trị
Launched map tasks / reduce tasks	4 / 1
Map input records	48,749
Map output records	462,618
Map output bytes	3,408,811
Map output materialized bytes = Reduce shuffle bytes	4,334,071
Reduce input groups = Reduce output records	20,967
HDFS bytes written (`part-00000`)	226,341
CPU time spent	7,010 ms
Thời gian lệnh (`real`)	23.572 s

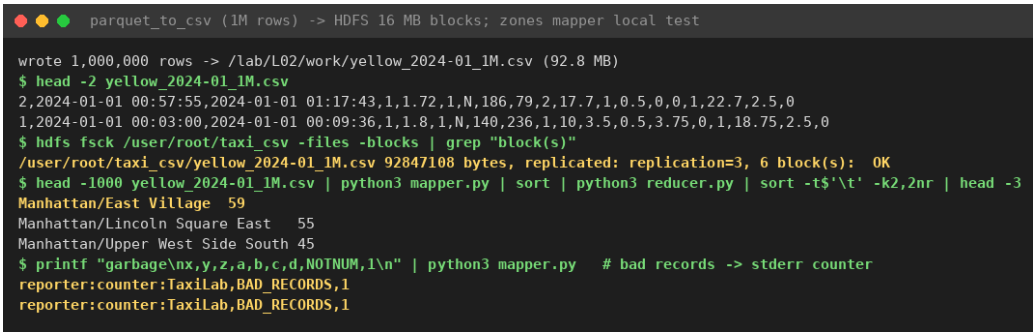


Hình 2. Counter cuối job `wordcount` trên YARN: 4 map task, `Reduce input groups=20967` và top 5 từ đọc từ HDFS.

2.3 Chuẩn bị dữ liệu taxi và map-side join

Tệp CSV được đưa lên bằng `hdfs dfs -D dfs.blocksize=16m -put` để job có nhiều split hơn. Số block dự kiến là $\lceil 92{,}847{,}108/16{,}777{,}216 \rceil = \lceil 5.53 \rceil = 6$, và `hdfs fsck` báo đúng `92847108 bytes, replicated: replication=3, 6 block(s): OK`.

Mapper khu vực nạp `taxi_zone_lookup.csv` vào một `dict` rồi đổi `PULocationID` thành `Borough/Zone` ngay khi đọc từng chuyến, nên bảng lớn không phải shuffle theo khóa nối. Dòng hỏng không làm mapper dừng mà được đếm bằng cách ghi `reporter:counter:TaxiLab,BAD_RECORDS,1` ra stderr. Lúc thử cục bộ, mapper chưa đọc được bảng tra cứu vì `-files` chỉ có tác dụng trên cluster, nên chúng tôi chép `taxi_zone_lookup.csv` vào thư mục hiện hành để giả lập distributed cache. Trên 1,000 dòng đầu, ba khu vực nhiều chuyến nhất là `Manhattan/East Village 59`, `Manhattan/Lincoln Square East 55` và `Manhattan/Upper West Side South 45`. Khi đưa vào hai dòng hỏng (một dòng `garbage`, một dòng có `PULocationID` không phải số), mapper ghi đúng hai dòng counter ra stderr (Hình 3). Dữ liệu thật không có dòng hỏng nào, nên nhóm `TaxiLab` không xuất hiện trong các job trên cluster và `Map input records` bằng `Map output records`.



Hình 3. Chuyển Parquet sang CSV, 6 block trên HDFS, kiểm thử cục bộ map-side join trên 1,000 dòng và counter `BAD_RECORDS`.

2.4 Có và không có combiner

Combiner là một reducer thu nhỏ chạy trên đầu ra từng map task trước shuffle. Hadoop có thể gọi nó 0, 1 hay nhiều lần, nên hàm gộp phải có tính kết hợp, giao hoán và giữ nguyên định dạng [1], [3]. Phép cộng thỏa điều kiện này nên reducer đếm được dùng lại làm combiner. Hai job chạy liền nhau, cùng đầu vào, cùng `-D mapreduce.job.reduces=2`, chỉ khác dòng `-combiner`:

```
hadoop jar $STREAM -D mapreduce.job.name=zones_combiner -D
 ↪  mapreduce.job.reduces=2 \
  -files mapper.py,reducer.py,/datasets/data/taxi/taxi_zone_lookup.csv \
  -mapper "python3 mapper.py" -combiner "python3 reducer.py" -reducer
   ↪  "python3 reducer.py" \
  -input /user/root/taxi_csv -output /user/root/zones_out_combiner
```

Bảng 4 gồm các counter đề bài yêu cầu cho `zones_nocombiner` (`job_1793355278127_0002`) và `zones_combiner` (`job_1793355278127_0003`); Bảng 5 gồm thêm vài counter lấy từ log đầy đủ. Wall-clock được tính bằng hiệu thời điểm giữa dòng log `Running job` và dòng `completed successfully`, không dùng lệnh `time`, nên không gồm phần khởi động client.

Bảng 4. So sánh counter giữa hai lần chạy không và có combiner.

Counter	Không combiner	Có combiner
Launched map tasks	6	6
Map output records	1,000,000	1,000,000
Combine input records	0	1,000,000
Combine output records	0	1,382
Map output materialized bytes	31,039,071	41,169
Reduce shuffle bytes	31,039,071	41,169
Reduce input records	1,000,000	1,382
Wall-clock time	30.3 s	33.3 s

Bảng 5. Counter bổ sung của hai lần chạy khu vực (trích từ `nocombiner.log` và `combiner.log`).

Counter	Không combiner	Có combiner
Map output bytes	29,038,999	29,038,999
Reduce input groups = Reduce output records	247	247
Spilled Records	2,000,000	2,764
FILE: Number of bytes written	64,607,796	2,614,896
FILE: Number of bytes read	31,039,011	41,109
Total time spent by all map tasks	17,287 ms	17,398 ms
Total time spent by all reduce tasks	13,013 ms	11,353 ms
CPU time spent	15,050 ms	14,020 ms
Shuffled Maps / Merged Map outputs	12 / 12	12 / 12
Killed reduce tasks	1	không xuất hiện

```
zones job: WITHOUT vs WITH combiner (-D mapreduce.job.reduces=2)

== /tmp/nocombiner.log
2026-10-30 10:25:37,187 INFO mapreduce.Job: Running job: job_1793355278127_0002
2026-10-30 10:26:07,488 INFO mapreduce.Job: Job job_1793355278127_0002 completed successfully
                Launched map tasks=6
                Launched reduce tasks=2
                Map output records=1000000
                Map output materialized bytes=31039071
                Combine input records=0
                Combine output records=0
                Reduce shuffle bytes=31039071
                Reduce input records=1000000
WALL nocombiner 30.3 s (10:25:37.187 Running job -> 10:26:07.488 completed)
== /tmp/combiner.log
2026-10-30 10:26:10,419 INFO mapreduce.Job: Running job: job_1793355278127_0003
2026-10-30 10:26:43,754 INFO mapreduce.Job: Job job_1793355278127_0003 completed successfully
                Launched map tasks=6
                Launched reduce tasks=2
                Map output records=1000000
                Map output materialized bytes=41169
                Combine input records=1000000
                Combine output records=1382
                Reduce shuffle bytes=41169
                Reduce input records=1382
WALL combiner 33.3 s (10:26:10.419 Running job -> 10:26:43.754 completed)
$ bash top_zones.sh /user/root/zones_out_combiner
Queens/JFK Airport      59777
Manhattan/Midtown Center        48750
Manhattan/Upper East Side South 47260
Manhattan/Upper East Side North 45867
Manhattan/Lincoln Square East   37424
Manhattan/Penn Station/Madison Sq West  37342
Manhattan/Midtown East  36421
Manhattan/Times Sq/Theatre District     36291
Queens/LaGuardia Airport        33240
Manhattan/Upper West Side South 30849
$ diff <(nocombiner output) <(combiner output)
IDENTICAL results
```

Hình 4. Trích các counter so sánh từ hai log và top 10 khu vực đón khách; `diff` giữa hai đầu ra cho kết quả IDENTICAL.

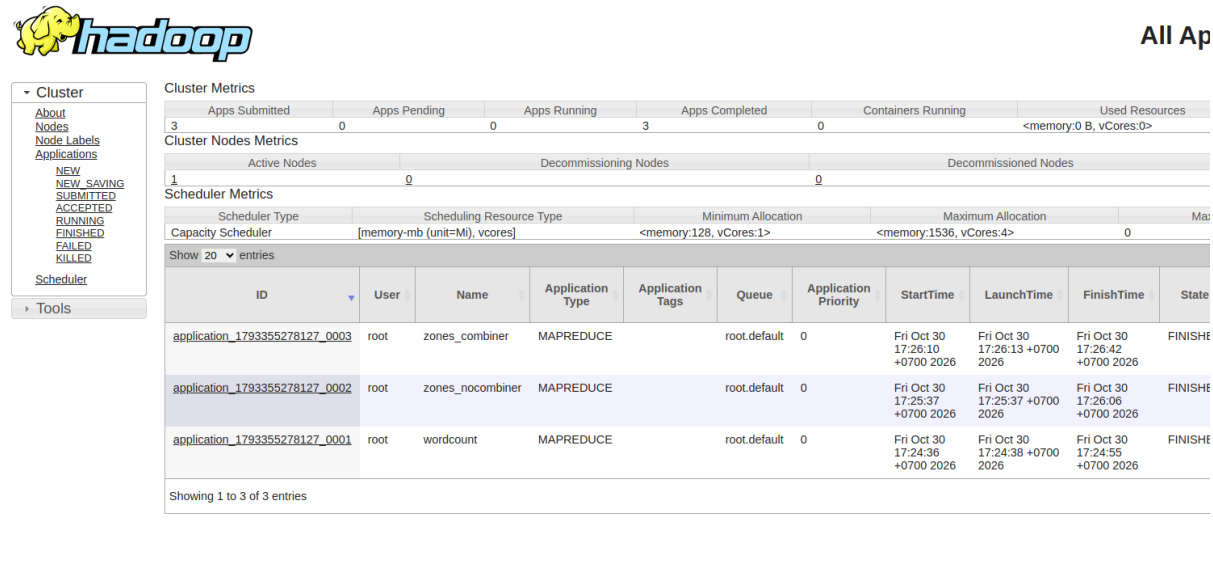
`Map output bytes` không đổi, tức mapper sinh cùng lượng dữ liệu ở cả hai lần; combiner thu gọn nó trước khi ghi đĩa và shuffle. Lượng shuffle giảm $31{,}039{,}071 / 41{,}169 \approx$ **754 lần** (khoảng 99.9 %), số bản ghi tràn đĩa và số byte ghi đĩa cục bộ giảm cùng cỡ. Cả hai lần đều có 247 khu vực vào reducer và `diff` hai đầu ra rỗng (Hình 4). Ba khu vực đứng đầu là `Queens/JFK Airport 59777`, `Manhattan/Midtown Center 48750` và `Manhattan/Upper East Side South 47260`.

Riêng thời gian thì ngược với dự đoán: bản có combiner chậm hơn 3.0 s. Tổng thời gian map gần như không đổi, chỉ thời gian reduce giảm. Lần chạy không combiner có `Killed reduce tasks=1`; đây là một attempt reduce bị hủy, cả hai reduce task vẫn chạy xong nên kết quả không bị ảnh hưởng.

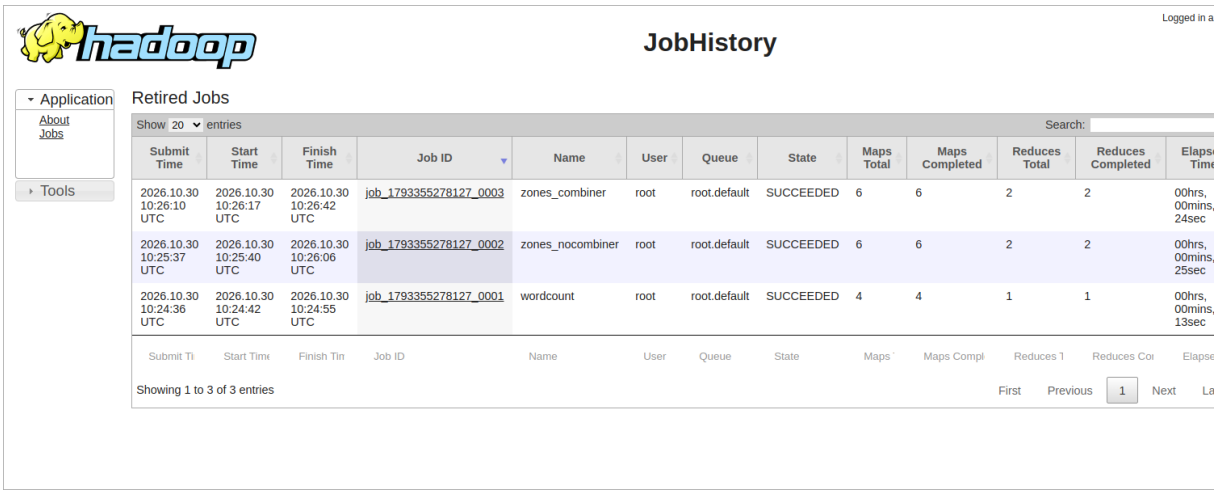
2.5 Giao diện YARN, JobHistory và CLI

ResourceManager UI liệt kê 3 ứng dụng đã xong (`wordcount`, `zones_nocombiner`, `zones_combiner`), 1 Active Node, Capacity Scheduler và giới hạn container như ở

Bảng 1 (Hình 5). Danh sách job trên JobHistory ở Hình 6.

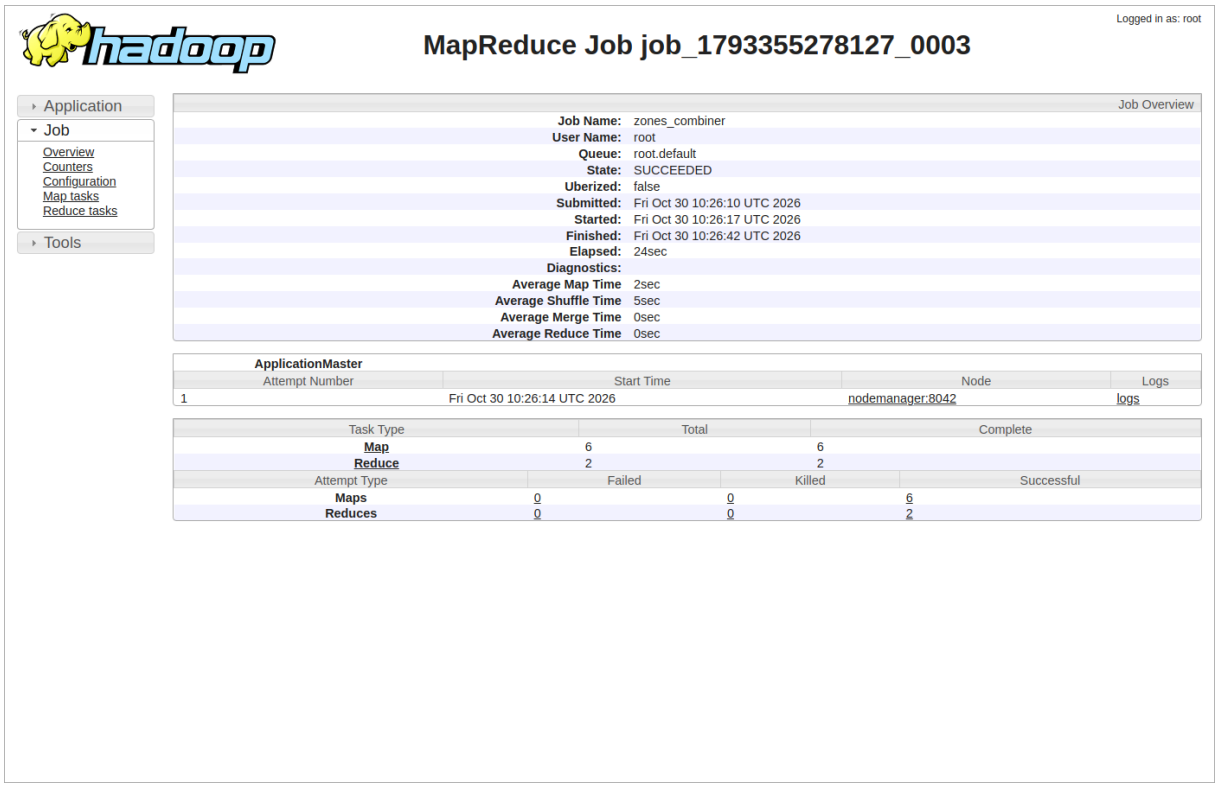


Hình 5. YARN ResourceManager UI (cổng 8088): ba ứng dụng MapReduce đã hoàn thành và các chỉ số cluster.



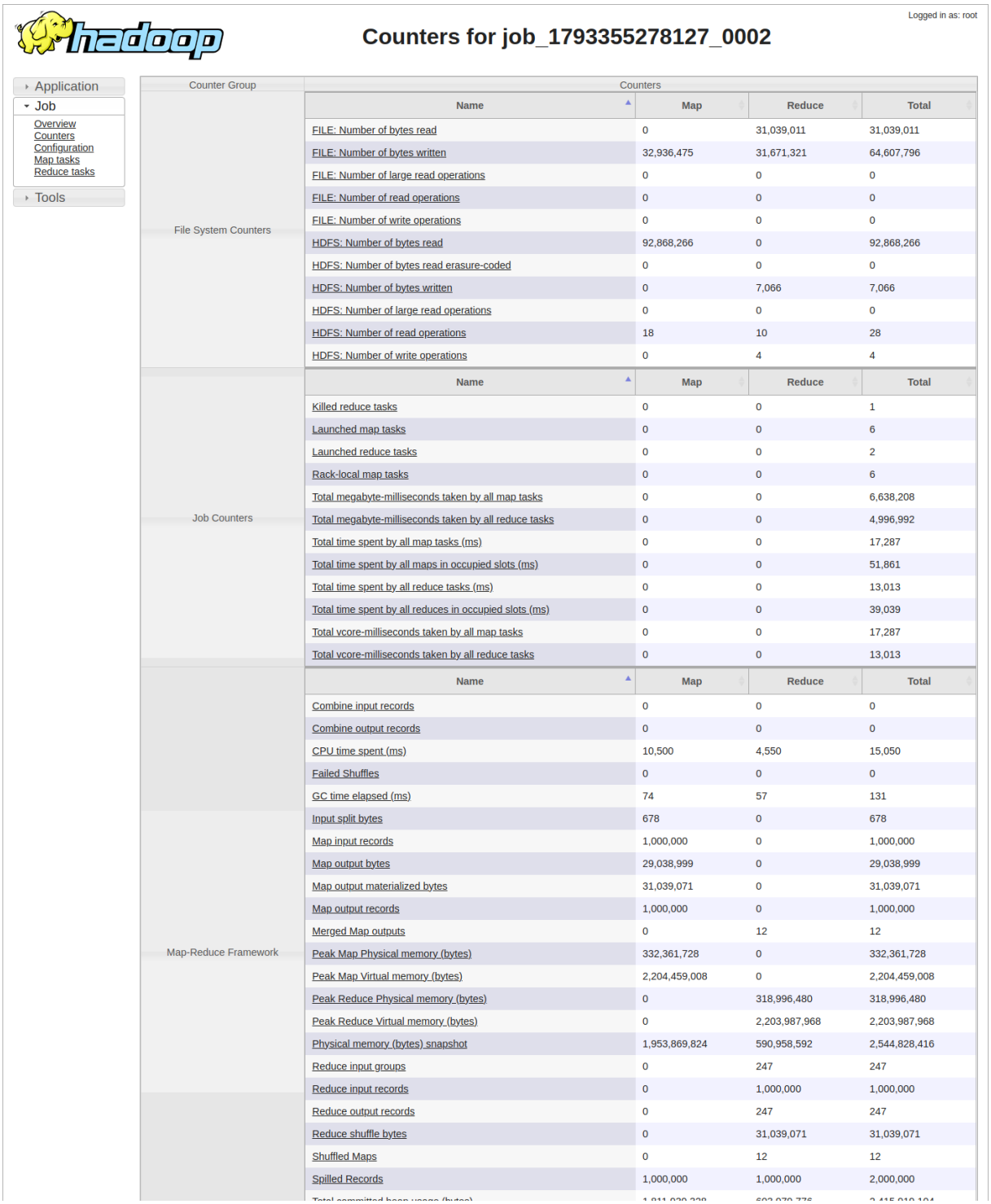
Hình 6. JobHistory UI (cổng 19888): danh sách các job đã hoàn thành.

Trang tổng quan của `job_1793355278127_0003` (Hình 7) ghi SUCCEEDED, không chạy chế độ uber, Elapsed 24 s, map trung bình 2 s, shuffle trung bình 5 s; cả 6 map và 2 reduce đều thành công, không attempt nào lỗi hay bị hủy. Elapsed ngắn hơn wall-clock ở Bảng 4 vì wall-clock còn gồm thời gian chờ ApplicationMaster khởi động.



Hình 7. JobHistory UI: tổng quan job `zones_combiner` với 6 map task, 2 reduce task và thời gian từng pha.

Trang Counters của hai job (Hình 8, Hình 9) cho đúng các giá trị in trên console.



Logged in as: root

Counters for job_1793355278127_0002

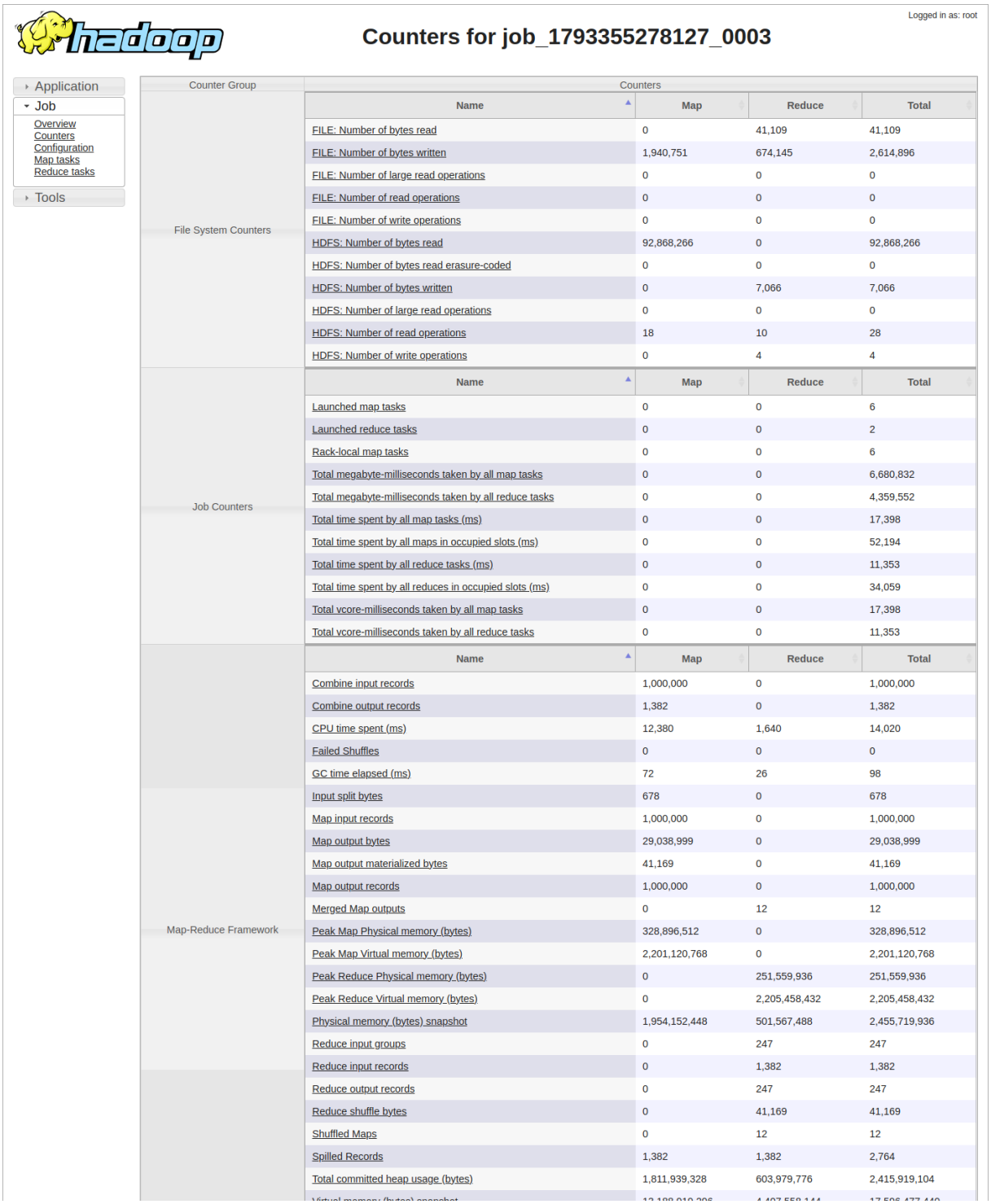
- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	31,039,011	31,039,011
	FILE: Number of bytes written	32,936,475	31,671,321	64,607,796
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4

Counter Group	Name	Map	Reduce	Total
Job Counters	Killed reduce tasks	0	0	1
	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,638,208
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,996,992
	Total time spent by all map tasks (ms)	0	0	17,287
	Total time spent by all maps in occupied slots (ms)	0	0	51,861
	Total time spent by all reduce tasks (ms)	0	0	13,013
	Total time spent by all reduces in occupied slots (ms)	0	0	39,039
	Total vcore-milliseconds taken by all map tasks	0	0	17,287
	Total vcore-milliseconds taken by all reduce tasks	0	0	13,013

Counter Group	Name	Map	Reduce	Total
Map-Reduce Framework	Combine input records	0	0	0
	Combine output records	0	0	0
	CPU time spent (ms)	10,500	4,550	15,050
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	74	57	131
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	31,039,071	0	31,039,071
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	332,361,728	0	332,361,728
	Peak Map Virtual memory (bytes)	2,204,459,008	0	2,204,459,008
	Peak Reduce Physical memory (bytes)	0	318,996,480	318,996,480
	Peak Reduce Virtual memory (bytes)	0	2,203,987,968	2,203,987,968
	Physical memory (bytes) snapshot	1,953,869,824	590,958,592	2,544,828,416
	Reduce input groups	0	247	247
	Reduce input records	0	1,000,000	1,000,000
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	31,039,071	31,039,071
	Shuffled Maps	0	12	12
	Spilled Records	1,000,000	1,000,000	2,000,000

Hình 8. JobHistory UI: trang Counters của job không combiner.



Logged in as: root

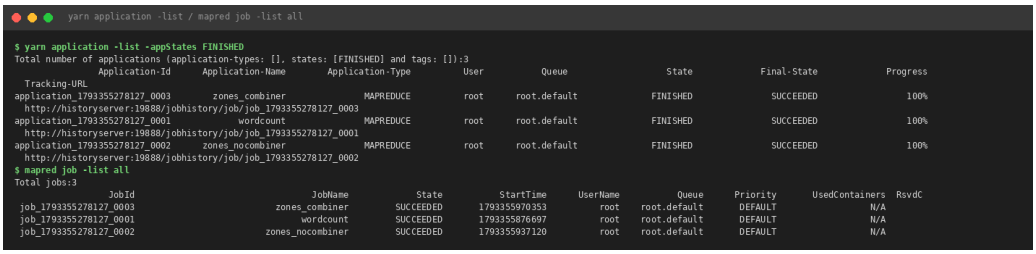
Counters for job_1793355278127_0003

- Application
- Job
  - Overview
  - Counters
  - Configuration
  - Map tasks
  - Reduce tasks
- Tools

Counter Group	Name	Map	Reduce	Total
File System Counters	FILE: Number of bytes read	0	41,109	41,109
	FILE: Number of bytes written	1,940,751	674,145	2,614,896
	FILE: Number of large read operations	0	0	0
	FILE: Number of read operations	0	0	0
	FILE: Number of write operations	0	0	0
	HDFS: Number of bytes read	92,868,266	0	92,868,266
	HDFS: Number of bytes read erasure-coded	0	0	0
	HDFS: Number of bytes written	0	7,066	7,066
	HDFS: Number of large read operations	0	0	0
	HDFS: Number of read operations	18	10	28
	HDFS: Number of write operations	0	4	4
Job Counters	Launched map tasks	0	0	6
	Launched reduce tasks	0	0	2
	Rack-local map tasks	0	0	6
	Total megabyte-milliseconds taken by all map tasks	0	0	6,680,832
	Total megabyte-milliseconds taken by all reduce tasks	0	0	4,359,552
	Total time spent by all map tasks (ms)	0	0	17,398
	Total time spent by all maps in occupied slots (ms)	0	0	52,194
	Total time spent by all reduce tasks (ms)	0	0	11,353
	Total time spent by all reduces in occupied slots (ms)	0	0	34,059
	Total vcore-milliseconds taken by all map tasks	0	0	17,398
	Total vcore-milliseconds taken by all reduce tasks	0	0	11,353
Map-Reduce Framework	Combine input records	1,000,000	0	1,000,000
	Combine output records	1,382	0	1,382
	CPU time spent (ms)	12,380	1,640	14,020
	Failed Shuffles	0	0	0
	GC time elapsed (ms)	72	26	98
	Input split bytes	678	0	678
	Map input records	1,000,000	0	1,000,000
	Map output bytes	29,038,999	0	29,038,999
	Map output materialized bytes	41,169	0	41,169
	Map output records	1,000,000	0	1,000,000
	Merged Map outputs	0	12	12
	Peak Map Physical memory (bytes)	328,896,512	0	328,896,512
	Peak Map Virtual memory (bytes)	2,201,120,768	0	2,201,120,768
	Peak Reduce Physical memory (bytes)	0	251,559,936	251,559,936
	Peak Reduce Virtual memory (bytes)	0	2,205,458,432	2,205,458,432
	Physical memory (bytes) snapshot	1,954,152,448	501,567,488	2,455,719,936
	Reduce input groups	0	247	247
	Reduce input records	0	1,382	1,382
	Reduce output records	0	247	247
	Reduce shuffle bytes	0	41,169	41,169
	Shuffled Maps	0	12	12
	Spilled Records	1,382	1,382	2,764
	Total committed heap usage (bytes)	1,811,939,328	603,979,776	2,415,919,104

Hình 9. JobHistory UI: trang Counters của job có combiner.

Trên dòng lệnh, `yarn application -list -appStates FINISHED` cho 3 ứng dụng có Final-State SUCCEEDED, với Tracking-URL trỏ tới `historyserver:19888`, và `mapred job -list all` báo `Total jobs:3` (Hình 10).



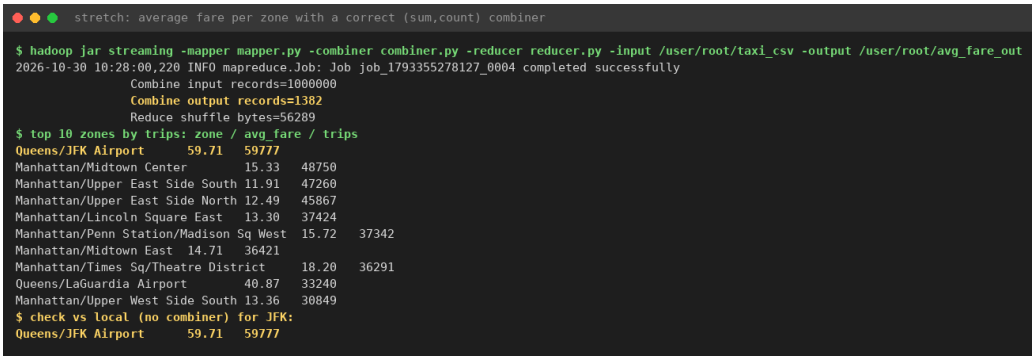
Hình 10. Kiểm tra các job đã hoàn thành bằng `yarn application -list` và `mapred job -list all`.

2.6 Mở rộng: giá cước trung bình theo khu vực

Mapper phát `zone\t<fare>,1`, trong đó `fare_amount` là cột thứ 10. Combiner cộng dồn cặp `(sum, count)` và xuất đúng định dạng nó nhận vào; reducer cộng tiếp và chỉ chia `sum/count` ở bước cuối. Job `job_1793355278127_0004` chạy xong với `Combine input records=1000000`, `Combine output records=1382` và `Reduce shuffle bytes=56289` (Bảng 6, Hình 11).

Bảng 6. Giá cước trung bình (`fare_amount`) của 10 khu vực đón khách nhiều chuyến nhất.

Khu vực	Giá cước trung bình	Số chuyến
Queens/JFK Airport	59.71	59,777
Manhattan/Midtown Center	15.33	48,750
Manhattan/Upper East Side South	11.91	47,260
Manhattan/Upper East Side North	12.49	45,867
Manhattan/Lincoln Square East	13.30	37,424
Manhattan/Penn Station/Madison Sq West	15.72	37,342
Manhattan/Midtown East	14.71	36,421
Manhattan/Times Sq/Theatre District	18.20	36,291
Queens/LaGuardia Airport	40.87	33,240
Manhattan/Upper West Side South	13.36	30,849



Hình 11. Job tính giá cước trung bình theo khu vực với combiner `(sum, count)` và đối chiếu JFK với bản chạy không combiner.

Tính lại JFK bằng bản cục bộ không combiner cũng được 59.71 trên 59,777 chuyến, và số chuyến

từng khu vực khớp với job đếm ở Mục 2.4. Số bản ghi shuffle vẫn là 1,382 nhưng số byte nhiều hơn vì mỗi giá trị mang thêm một tổng số thực. Hai sân bay có giá trung bình cao hơn hẳn các khu vực Manhattan, do chuyến từ sân bay thường dài hơn.

3 Trả lời câu hỏi checkpoint

3.1 Trong đường ống cục bộ, lệnh nào làm việc của shuffle, và thiếu nó thì sao?

Lệnh `sort`. Trên cluster, shuffle phân vùng đầu ra map theo khóa rồi merge-sort ở reducer, nên các bản ghi cùng khóa luôn nằm liền nhau [3]; `sort` cho đúng bảo đảm đó. Không có nó, reducer gặp cùng một từ ở nhiều đoạn rời nhau và in ra nhiều lần với số đếm cục bộ, như các dòng `the` ở Hình 1.

3.2 Vì sao reducer Streaming phải tự giữ `current_key`?

Streaming chỉ đưa cho reducer các dòng văn bản đã sắp xếp; nhóm `(key, Iterable<values>)` chỉ có trong API Java [2]. Reducer Python phải so khóa với `current_key`, in tổng khi khóa đổi và nhớ in khóa cuối sau vòng lặp (Phụ lục). Quên bước cuối thì mất từ cuối cùng. Reducer của bài không mắc lỗi này: `Reduce output records` bằng `Reduce input groups` ở Bảng 3.

3.3 Job taxi có bao nhiêu map task, bao nhiêu reduce task, và vì sao?

Job taxi có 6 map task vì tệp CSV nằm trong 6 block 16 MB và mỗi block là một split (log ghi `number of splits:6`, Mục 2.3); job đếm từ chỉ có 4 vì mỗi cuốn sách nhỏ hơn một block. Số reduce task là 2 vì chúng tôi đặt `-D mapreduce.job.reduces=2`. Con số này do người dùng chọn chứ không suy ra từ dữ liệu, job đếm từ để mặc định là 1. Với 2 reducer, mỗi map task tạo hai phân vùng, nên `Shuffled Maps` bằng $6 \times 2 = 12$ (Bảng 5). NodeManager chỉ chạy hai task 384 MB cùng lúc nên 6 map chạy thành nhiều đợt; log thể hiện qua tiến độ map tăng dần 17 %, 33 %, 50 %, 67 %, 83 % rồi 100 %.

3.4 Combiner giảm `Reduce shuffle bytes` bao nhiêu lần, và vì sao đầu ra combiner xấp xỉ số khu vực nhân số map task?

Khoảng 754 lần (Mục 2.4). Combiner chạy riêng trên đầu ra từng map task, và mỗi task chỉ xuất một bản ghi cho mỗi khu vực nó gặp. Toàn bộ dữ liệu có 247 khu vực, còn mỗi task ra trung bình $1{,}382/6 \approx 230$ bản ghi, tức mỗi split gặp gần hết các khu vực. Chỉ các tổng cục bộ này đi qua shuffle, nên `Reduce input records` cũng chỉ còn bằng `Combine output records` (Bảng 4).

3.5 Vì sao combiner không làm job nhanh hơn ở đây, và khi nào nó có ích rõ?

Trên một máy, shuffle chỉ là chép dữ liệu qua đĩa cục bộ hoặc loopback, gần như không tốn gì, trong khi combiner thêm một tiến trình Python phải đọc, phân tích và sắp xếp lại toàn bộ bản ghi đầu ra map. Tổng thời gian map không giảm, còn phần tiết kiệm ở reduce, khoảng 1.7 s (Bảng 5), không bù được thời gian khởi động và dao động khi lập lịch container. Mỗi cấu hình mới chạy một lần nên chênh lệch 3.0 s chưa đủ để kết luận combiner làm chậm; bản tham chiếu của đề bài cũng ghi khoảng 28 s và 37 s. Trên cluster thật, shuffle đi qua mạng và mỗi reducer phải merge hàng GB từ hàng trăm mapper. Khi đó việc bỏ được gần hết lượng shuffle và số bản ghi tràn đĩa sẽ lớn hơn nhiều so với chi phí CPU của combiner.

3.6 Có dùng lại reducer "trung bình" làm combiner được không?

Không được. Trung bình của các trung bình sai khi các phân vùng có số bản ghi khác nhau: trung bình của $\{10\}$ và $\{20, 30\}$ là $(10 + 20 + 30)/3 = 20$, còn trung bình hai trung bình cục bộ là $(10 + 25)/2 = 17.5$. Reducer trung bình còn xuất ra một số thực, khác định dạng đầu vào, nên không gộp tiếp được khi Hadoop gọi combiner nhiều lần. Cách đúng là để combiner phát `(sum, count)` và chỉ reducer mới chia. Phần mở rộng làm theo cách này và kết quả JFK khớp với bản không combiner (Mục 2.6).

3.7 Map-side join trong bài hoạt động thế nào, và khi nào nó không còn hợp?

Bảng `taxi_zone_lookup.csv` được gửi tới mọi task qua `-files`, mỗi mapper nạp vào bộ nhớ và tra cứu ngay khi đọc chuyến (Mục 2.3), nên phép nối xong ở pha map. Cách này không còn hợp khi bảng "nhỏ" không vừa thoải mái trong bộ nhớ task (ở đây mỗi task chỉ có 384 MB), hoặc khi gửi bảng tới mọi task tốn hơn cả shuffle. Khi đó phải dùng reduce-side join, tức shuffle cả hai tập theo khóa nối [3]. Spark gọi ý tưởng thứ nhất là broadcast join, sẽ gặp lại ở L06.

3.8 Counter `BAD_RECORDS` hiện ở đâu, và vì sao nên đếm dòng hỏng thay vì cho job dừng?

Counter nằm trong job counters, nhóm `TaxiLab`: phần in ra console cuối job và mục Counters trên JobHistory UI. Nó chỉ hiện khi có ít nhất một dòng hỏng; dữ liệu trong bài không có dòng hỏng nên chỉ thấy được ở phép thử cục bộ (Hình 3). Một job xử lý hàng tỷ dòng không nên chết vì vài dòng lỗi và bỏ phí mọi công việc đã làm. Đếm dòng hỏng vẫn cho thấy vấn đề chất lượng dữ liệu, và có thể đặt ngưỡng để cảnh báo hoặc dừng job khi số dòng hỏng quá nhiều.

4 Kết luận

Bài đã hoàn thành các task của đề: mapper và reducer đếm từ cho cùng top 5 khi chạy cục bộ và trên YARN, mapper khu vực có map-side join và counter dòng hỏng, bảng counter so sánh có và không có combiner, kiểm tra job trên hai giao diện web và CLI, cùng phần mở rộng giá cước trung bình với combiner (`sum, count`). Combiner giảm dữ liệu shuffle khoảng 754 lần mà không đổi kết quả, nhưng không làm job nhanh hơn; chúng tôi cho rằng lý do nằm ở môi trường một máy chứ không phải ở combiner. Hạn chế chính là mọi thứ chạy trên một NodeManager, shuffle không đi qua mạng thật, mỗi cấu hình chỉ chạy một lần, và `BAD_RECORDS` mới được kiểm chứng ở chế độ cục bộ.

Tài liệu tham khảo

[1] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in *Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.

[2] Apache Software Foundation, “Hadoop Streaming,” Apache Hadoop documentation. https://hadoop.apache.org/docs/stable/hadoop-streaming/HadoopStreaming.html

[3] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA, USA: O’Reilly Media, 2015.

Phụ lục: Trích đoạn mã nguồn

Mã đầy đủ và log của từng task nằm trong `submission/L02/` (Bảng 7).

Bảng 7. Danh mục tệp nộp kèm trong `submission/L02/`.

Tệp	Nội dung
`wc_mapper.py`, `wc_reducer.py`, `zones_mapper.py`	Mapper/reducer đếm từ, mapper khu vực
`stretch_avg_fare/`	Mapper, combiner, reducer giá cước trung bình
`01_local_pipe.txt` đến `06_stretch_avg_fare.txt`	Log từng task
`nocombiner.log`, `combiner.log`	Log đầy đủ hai job khu vực

Reducer đếm từ (`wc_reducer.py`) tự nhận ra chỗ đổi khóa và in thêm khóa cuối cùng sau vòng lặp.

```
current_word, current_count = None, 0
```

```
for line in sys.stdin:
    word, _, count = line.rstrip("\n").partition("\t")
    count = int(count)
    if word == current_word:
        current_count += count
    else:
        if current_word is not None:
            print(f"{current_word}\t{current_count}")
        current_word, current_count = word, count
if current_word is not None:  # the last word's total
    print(f"{current_word}\t{current_count}")
```

Mapper khu vực (`zones_mapper.py`) nạp bảng tra cứu, báo dòng hỏng qua stderr và phát `zone\t1`.

```
zones = {}
if os.path.exists("taxi_zone_lookup.csv"):
    with open("taxi_zone_lookup.csv", newline="") as f:
        for row in csv.DictReader(f):
            zones[row["LocationID"]] = f"{row['Borough']}/{row['Zone']}"
for line in sys.stdin:
    fields = line.rstrip("\n").split(",")
    if len(fields) <= PU_COL or not fields[PU_COL].strip().isdigit():
        sys.stderr.write("reporter:counter:TaxiLab,BAD_RECORDS,1\n")
        continue
    pu = fields[PU_COL].strip()
    print(f"{zones.get(pu, pu)}\t1")
```